

Section 8.3.2 - Fine-tuning a pretrained model

December 2, 2025

1 Fine-tuning a pretrained model

- Tune = adjust & fine = very small, delicate
- Adjust more abstract representations to become more relevant to the problem at hand
- Use small learning rate - limit the magnitude of modifications
- Train the classifier before fine-tuning

- 1) Add new classifier
- 2) Freeze the base
- 3) Train the classifier (feature extraction)
- 4) Unfreeze a few top layers of the convolutional base
- 5) Train the new classifier & these top layers

1.1 Preparing the datasets

```
[1]: from pathlib import Path
from tensorflow.keras.utils import image_dataset_from_directory

new_base_dir = Path("cats_vs_dogs_small")

train_dataset = image_dataset_from_directory(
    new_base_dir / "train",
    image_size=(180, 180),
    batch_size=32)
validation_dataset = image_dataset_from_directory(
    new_base_dir / "validation",
    image_size=(180, 180),
    batch_size=32)
test_dataset = image_dataset_from_directory(
    new_base_dir / "test",
    image_size=(180, 180),
    batch_size=32)
```

Found 2000 files belonging to 2 classes.

Found 1000 files belonging to 2 classes.

Found 2000 files belonging to 2 classes.

1.2 Instantiating the VGG16 convolutional base

```
[2]: from tensorflow import keras
from tensorflow.keras import layers

conv_base = keras.applications.vgg16.VGG16(
    weights="imagenet",
    include_top=False)
conv_base.trainable = False

# summary of the convolutional base
conv_base.summary()
```

Model: "vgg16"

Layer (type)	Output Shape	Param #
input_1 (InputLayer)	[(None, None, None, 3)]	0
block1_conv1 (Conv2D)	(None, None, None, 64)	1792
block1_conv2 (Conv2D)	(None, None, None, 64)	36928
block1_pool (MaxPooling2D)	(None, None, None, 64)	0
block2_conv1 (Conv2D)	(None, None, None, 128)	73856
block2_conv2 (Conv2D)	(None, None, None, 128)	147584
block2_pool (MaxPooling2D)	(None, None, None, 128)	0
block3_conv1 (Conv2D)	(None, None, None, 256)	295168
block3_conv2 (Conv2D)	(None, None, None, 256)	590080
block3_conv3 (Conv2D)	(None, None, None, 256)	590080
block3_pool (MaxPooling2D)	(None, None, None, 256)	0
block4_conv1 (Conv2D)	(None, None, None, 512)	1180160
block4_conv2 (Conv2D)	(None, None, None, 512)	2359808
block4_conv3 (Conv2D)	(None, None, None, 512)	2359808
block4_pool (MaxPooling2D)	(None, None, None, 512)	0
block5_conv1 (Conv2D)	(None, None, None, 512)	2359808

```

block5_conv2 (Conv2D)          (None, None, None, 512)    2359808

block5_conv3 (Conv2D)          (None, None, None, 512)    2359808

block5_pool (MaxPooling2D)     (None, None, None, 512)    0

=====
Total params: 14,714,688
Trainable params: 0
Non-trainable params: 14,714,688
-----

```

1.3 Freezing all layers until the fourth from the last

```

[3]: conv_base.trainable = True
     for layer in conv_base.layers[:-4]:
         layer.trainable = False

```

1.4 The model to fine-tune

```

[4]: # Hide the warning messages
     import tensorflow as tf
     import logging

     tf.get_logger().setLevel(logging.ERROR)

     data_augmentation = keras.Sequential(
         [
             layers.RandomFlip("horizontal"),
             layers.RandomRotation(0.1),
             layers.RandomZoom(0.2),
         ]
     )

     inputs = keras.Input(shape=(180, 180, 3))
     x = data_augmentation(inputs)
     x = keras.applications.vgg16.preprocess_input(x)
     x = conv_base(x)
     x = layers.Flatten()(x)
     x = layers.Dense(256)(x)
     x = layers.Dropout(0.5)(x)
     outputs = layers.Dense(1, activation="sigmoid")(x)
     model = keras.Model(inputs, outputs)

     # summary of the fine-tuning model
     model.summary()

```

Model: "model"

Layer (type)	Output Shape	Param #
input_2 (InputLayer)	[(None, 180, 180, 3)]	0
sequential (Sequential)	(None, 180, 180, 3)	0
tf.__operators__.getitem (SlicingOpLambda)	(None, 180, 180, 3)	0
tf.nn.bias_add (TFOpLambda)	(None, 180, 180, 3)	0
vgg16 (Functional)	(None, None, None, 512)	14714688
flatten (Flatten)	(None, 12800)	0
dense (Dense)	(None, 256)	3277056
dropout (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 1)	257

Total params: 17,992,001
Trainable params: 10,356,737
Non-trainable params: 7,635,264

```
[5]: model.compile(loss="binary_crossentropy",
                  optimizer=keras.optimizers.RMSprop(learning_rate=1e-5),
                  metrics=["accuracy"])

callbacks = [
    keras.callbacks.ModelCheckpoint(
        filepath="fine_tuning.h5",
        save_best_only=True,
        monitor="val_loss")
]

history = model.fit(
    train_dataset,
    epochs=30,
    validation_data=validation_dataset,
    callbacks=callbacks)
```

Epoch 1/30

63/63 [=====] - 17s 183ms/step - loss: 3.5112 -

accuracy: 0.7530 - val_loss: 0.6965 - val_accuracy: 0.9220
Epoch 2/30
63/63 [=====] - 10s 156ms/step - loss: 1.1280 -
accuracy: 0.8645 - val_loss: 0.4549 - val_accuracy: 0.9500
Epoch 3/30
63/63 [=====] - 10s 154ms/step - loss: 0.6444 -
accuracy: 0.9080 - val_loss: 0.3530 - val_accuracy: 0.9580
Epoch 4/30
63/63 [=====] - 10s 154ms/step - loss: 0.4366 -
accuracy: 0.9300 - val_loss: 0.2820 - val_accuracy: 0.9590
Epoch 5/30
63/63 [=====] - 10s 156ms/step - loss: 0.2827 -
accuracy: 0.9395 - val_loss: 0.2727 - val_accuracy: 0.9640
Epoch 6/30
63/63 [=====] - 10s 156ms/step - loss: 0.2185 -
accuracy: 0.9490 - val_loss: 0.2403 - val_accuracy: 0.9660
Epoch 7/30
63/63 [=====] - 10s 156ms/step - loss: 0.2046 -
accuracy: 0.9520 - val_loss: 0.2256 - val_accuracy: 0.9690
Epoch 8/30
63/63 [=====] - 10s 157ms/step - loss: 0.1087 -
accuracy: 0.9680 - val_loss: 0.2255 - val_accuracy: 0.9680
Epoch 9/30
63/63 [=====] - 10s 154ms/step - loss: 0.1384 -
accuracy: 0.9655 - val_loss: 0.2293 - val_accuracy: 0.9660
Epoch 10/30
63/63 [=====] - 10s 155ms/step - loss: 0.1012 -
accuracy: 0.9700 - val_loss: 0.2119 - val_accuracy: 0.9670
Epoch 11/30
63/63 [=====] - 10s 152ms/step - loss: 0.0613 -
accuracy: 0.9800 - val_loss: 0.2363 - val_accuracy: 0.9740
Epoch 12/30
63/63 [=====] - 10s 152ms/step - loss: 0.0824 -
accuracy: 0.9780 - val_loss: 0.2679 - val_accuracy: 0.9640
Epoch 13/30
63/63 [=====] - 10s 154ms/step - loss: 0.0630 -
accuracy: 0.9850 - val_loss: 0.2433 - val_accuracy: 0.9710
Epoch 14/30
63/63 [=====] - 10s 155ms/step - loss: 0.0921 -
accuracy: 0.9780 - val_loss: 0.2139 - val_accuracy: 0.9720
Epoch 15/30
63/63 [=====] - 10s 153ms/step - loss: 0.0515 -
accuracy: 0.9865 - val_loss: 0.2634 - val_accuracy: 0.9680
Epoch 16/30
63/63 [=====] - 10s 157ms/step - loss: 0.0633 -
accuracy: 0.9850 - val_loss: 0.2375 - val_accuracy: 0.9730
Epoch 17/30
63/63 [=====] - 10s 156ms/step - loss: 0.0405 -

```

accuracy: 0.9885 - val_loss: 0.2257 - val_accuracy: 0.9750
Epoch 18/30
63/63 [=====] - 10s 156ms/step - loss: 0.0544 -
accuracy: 0.9845 - val_loss: 0.2133 - val_accuracy: 0.9740
Epoch 19/30
63/63 [=====] - 10s 156ms/step - loss: 0.0204 -
accuracy: 0.9925 - val_loss: 0.2328 - val_accuracy: 0.9730
Epoch 20/30
63/63 [=====] - 10s 156ms/step - loss: 0.0375 -
accuracy: 0.9905 - val_loss: 0.2110 - val_accuracy: 0.9730
Epoch 21/30
63/63 [=====] - 10s 160ms/step - loss: 0.0301 -
accuracy: 0.9925 - val_loss: 0.2165 - val_accuracy: 0.9720
Epoch 22/30
63/63 [=====] - 10s 160ms/step - loss: 0.0253 -
accuracy: 0.9915 - val_loss: 0.2035 - val_accuracy: 0.9730
Epoch 23/30
63/63 [=====] - 10s 153ms/step - loss: 0.0364 -
accuracy: 0.9910 - val_loss: 0.2666 - val_accuracy: 0.9710
Epoch 24/30
63/63 [=====] - 10s 153ms/step - loss: 0.0199 -
accuracy: 0.9955 - val_loss: 0.2302 - val_accuracy: 0.9750
Epoch 25/30
63/63 [=====] - 10s 152ms/step - loss: 0.0247 -
accuracy: 0.9950 - val_loss: 0.2803 - val_accuracy: 0.9720
Epoch 26/30
63/63 [=====] - 10s 153ms/step - loss: 0.0145 -
accuracy: 0.9950 - val_loss: 0.3244 - val_accuracy: 0.9740
Epoch 27/30
63/63 [=====] - 10s 153ms/step - loss: 0.0151 -
accuracy: 0.9960 - val_loss: 0.2930 - val_accuracy: 0.9740
Epoch 28/30
63/63 [=====] - 10s 153ms/step - loss: 0.0426 -
accuracy: 0.9920 - val_loss: 0.2143 - val_accuracy: 0.9760
Epoch 29/30
63/63 [=====] - 10s 156ms/step - loss: 0.0110 -
accuracy: 0.9960 - val_loss: 0.2267 - val_accuracy: 0.9760
Epoch 30/30
63/63 [=====] - 10s 156ms/step - loss: 0.0206 -
accuracy: 0.9940 - val_loss: 0.2314 - val_accuracy: 0.9770

```

1.5 Plotting the training results

```

[6]: import matplotlib.pyplot as plt

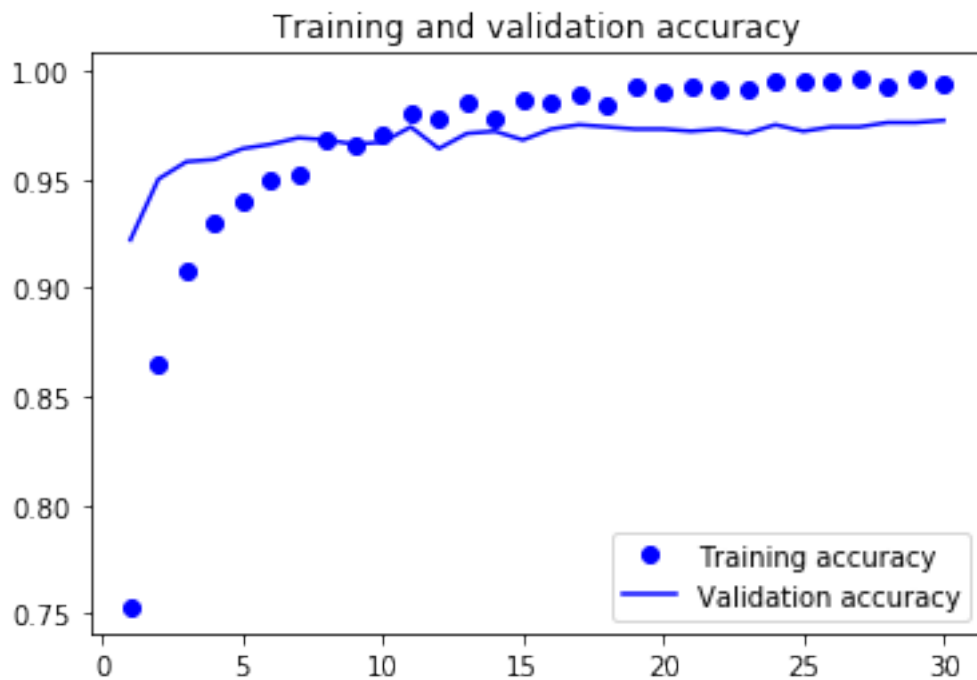
acc = history.history["accuracy"]
val_acc = history.history["val_accuracy"]

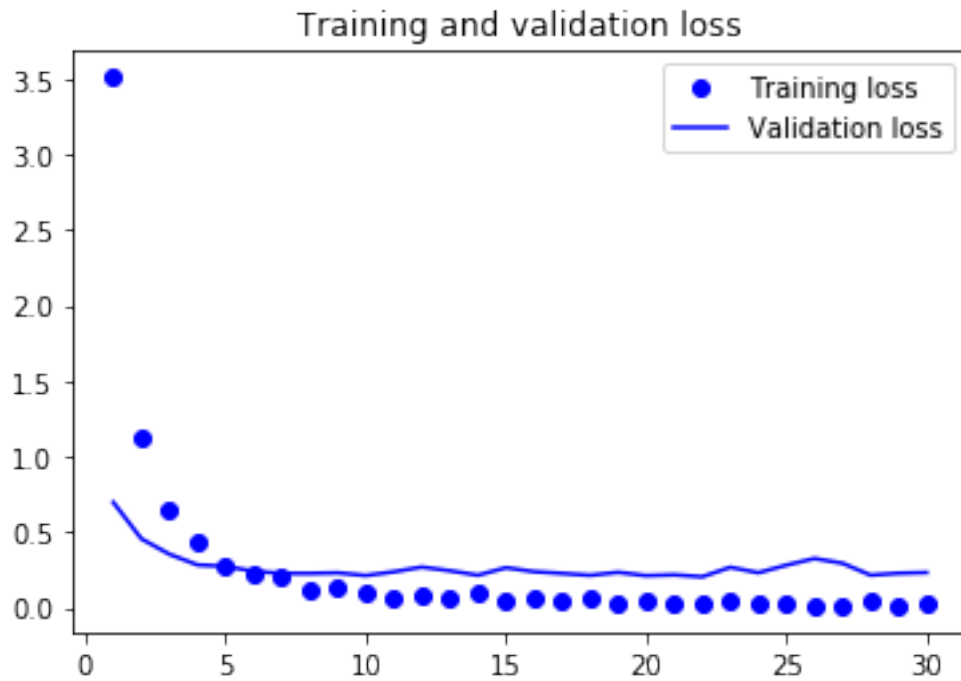
```

```

loss = history.history["loss"]
val_loss = history.history["val_loss"]
epochs = range(1, len(acc) + 1)
plt.plot(epochs, acc, "bo", label="Training accuracy")
plt.plot(epochs, val_acc, "b", label="Validation accuracy")
plt.title("Training and validation accuracy")
plt.legend()
plt.figure()
plt.plot(epochs, loss, "bo", label="Training loss")
plt.plot(epochs, val_loss, "b", label="Validation loss")
plt.title("Training and validation loss")
plt.legend()
plt.show()

```





1.6 Evaluation on the test dataset

```
[7]: model = keras.models.load_model("fine_tuning.h5")
test_loss, test_acc = model.evaluate(test_dataset)
print(f"Test accuracy: {test_acc:.3f}")
```

```
63/63 [=====] - 4s 63ms/step - loss: 0.1794 - accuracy:
0.9745
Test accuracy: 0.975
```